목차

1장: 머신러닝 개요 (Introduction to Machine Learning) 2
목차 . . . 2
1.1 머신러닝이란 . . . 2
1.1.1 정의 . . . 3
1.1.2 전통적 프로그래밍 vs 머신러닝 . . . 3
1.1.3 일상 속 머신러닝 사례 5가지 . . . 3
1.1.4 머신러닝이 필요한 이유 . . . 4
1.2 머신러닝의 역사 . . . 4
1.2.1 주요 마일스톤 연대표 . . . 4
1.2.2 AI 겨울과 부활 . . . 5
1.3 학습 유형 - 지도/비지도/강화학습 . . . 5
1.3.1 지도학습 (Supervised Learning) . . . 5
1.3.2 비지도학습 (Unsupervised Learning) . . . 5
1.3.3 강화학습 (Reinforcement Learning) . . . 6
1.3.4 세 가지 학습 유형 비교 . . . 6
1.4 ML vs DL vs AI 관계 . . . 6
1.4.1 계층 구조 . . . 6
1.4.2 ML vs DL 상세 비교 . . . 7
1.4.3 언제 무엇을 사용할 것인가 . . . 7
1.5 편향-분산 트레이드오프 . . . 7
1.5.1 기본 개념 . . . 7
1.5.2 수식 유도 . . . 7
1.5.3 직관적 설명 . . . 8
1.5.4 그래프 해석 . . . 8
1.5.5 모델별 편향-분산 특성 . . . 9
1.6 No Free Lunch 정리 . . . 9
1.6.1 정리의 내용 . . . 9
1.6.2 수학적 정의 . . . 9
1.6.3 핵심 가정과 한계 . . . 9
1.6.4 실무적 시사점 . . . 10
1.7 모델 평가 - 교차검증 전략 비교 . . . 10
1.7.1 왜 데이터를 나누는가? . . . 10
1.7.2 Hold-Out . . . 10
1.7.3 K-Fold 교차검증 . . . 10
1.7.4 Stratified K-Fold . . . 10
1.7.5 Leave-One-Out (LOO) . . . 10
1.7.6 네 가지 전략 비교 . . . 11
1.7.7 K값에 따른 편향-분산 트레이드오프 . . . 11
1.7.8 데이터 크기별 권장 전략 . . . 11
1.8 ML 파이프라인과 데이터 품질 . . . 11
1.8.1 머신러닝 파이프라인 . . . 11
1.8.2 데이터 품질의 중요성 . . . 12
1.8.3 피처 엔지니어링의 중요성 . . . 12
1.8.4 데이터 누출 (Data Leakage) 방지 . . . 12
1.9 핵심 논문 리뷰 . . . 12
1.9.1 Domingos (2012) - "A Few Useful Things to Know About Machine Learning" . . . 12
1.9.2 Mitchell (1997) - "Machine Learning" Ch1-2 . . . 13
1.9.3 Wolpert & Macready (1997) - "No Free Lunch Theorems for Optimization" . . . 13
1.9.4 Vapnik (1995) - "The Nature of Statistical Learning Theory" . . . 13
1.9.5 Hastie, Tibshirani, Friedman (2009) - "The Elements of Statistical Learning" Ch1-2 14

1.10 실습: 편향-분산 트레이드오프 구현 . . . 14
1.10.1 실습 목표 . . . 14
1.10.2 전체 코드 . . . 14
1.10.3 코드 해설 . . . 21
1.11 실습: NFL 정리 데모 . . . 21
1.11.1 실습 목표 . . . 21
1.11.2 전체 코드 . . . 21
1.11.3 코드 해설 . . . 25
1.12 실습: 교차검증 비교 . . . 26
1.12.1 실습 목표 . . . 26
1.12.2 전체 코드 . . . 26
1.12.3 코드 해설 . . . 31
1.13 응용사례 심화 . . . 32
1.13.1 의료 (Healthcare) . . . 32
1.13.2 금융 (Finance) . . . 32
1.13.3 제조 (Manufacturing) . . . 32
1.13.4 NLP (Natural Language Processing) . . . 33
1.14 핵심 요약표 + 복습 질문 10개 . . . 33
1.14.1 핵심 요약표 . . . 33
1.14.2 핵심 수식 정리 . . . 34
1.14.3 복습 질문 10개 . . . 34
참고문헌 . . . 35

1장: 머신러닝 개요 (Introduction to Machine Learning)

과목: 기계학습 **범위**: Chapter 1 - 머신러닝 개요 **핵심 키워드**: 머신러닝 정의, 학습 유형, 편향-분산 트레이드오프, NFL 정리, 교차검증, ML 파이프라인

목차

1. 머신러닝이란
2. 머신러닝의 역사
3. 학습 유형
4. ML vs DL vs AI 관계
5. 편향-분산 트레이드오프
6. No Free Lunch 정리
7. 모델 평가
8. ML 파이프라인과 데이터 품질
9. 핵심 논문 리뷰
10. 실습: 편향-분산 트레이드오프 구현
11. 실습: NFL 정리 데모
12. 실습: 교차검증 비교
13. 응용사례 심화
14. 핵심 요약표 + 복습 질문

1.1 머신러닝이란

1.1.1 정의

머신러닝(Machine Learning)이란 컴퓨터가 **명시적으로 프로그래밍하지 않아도** 데이터로부터 패턴을 학습하고, 이를 바탕으로 예측이나 결정을 내릴 수 있도록 하는 인공지능(AI)의 한 분야이다.

> **Arthur Samuel (1959)**: "Machine Learning is a field of study that gives computers the ability to learn without being explicitly programmed."
>
> **Tom Mitchell (1997)**: "A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."

Mitchell의 정의는 학습의 세 요소를 명확히 한다:

요소	의미	예시 (스팸 필터)
Task (T)	수행하고자 하는 작업	이메일의 스팸 여부 분류
Performance (P)	성능 측정 지표	정확하게 분류된 이메일 비율
Experience (E)	학습에 사용되는 데이터	사용자가 레이블링한 이메일 데이터

1.1.2 전통적 프로그래밍 vs 머신러닝

구분	전통적 프로그래밍	머신러닝
입력	데이터 + 규칙(Rules)	데이터 + 정답(Labels)
출력	결과(Answers)	규칙/모델(Model)
방식	사람이 규칙을 직접 코딩	데이터로부터 규칙을 자동 학습
예시	if-else 기반 스팸 필터	수만 건의 이메일로 학습한 스팸 필터

1.1.3 일상 속 머신러닝 사례 5가지

1) 추천 시스템 (Recommendation System)

넷플릭스, 유튜브, 스포티파이 등에서 사용자의 시청/청취 이력을 분석하여 개인화된 콘텐츠를 추천한다. 협업 필터링(Collaborative Filtering)과 콘텐츠 기반 필터링(Content-based Filtering)이 핵심 기법이다. 넷플릭스는 추천 알고리즘을 통해 사용자 시청 시간의 약 80%를 유도한다고 알려져 있다.

2) 자율주행 (Autonomous Driving)

테슬라, 웨이모 등의 자율주행 시스템은 카메라, 라이다, 레이더 센서 데이터를 실시간으로 처리하여 차선 인식, 객체 탐지, 경로 계획을 수행한다. 컴퓨터 비전(딥러닝 기반 CNN)과 강화학습이 핵심 기술이며, 수백만 km의 주행 데이터로 학습한다. 보행자, 차량, 신호등 등을 실시간으로 인식하고 주행 결정을 내린다.

3) 의료 진단 (Medical Diagnosis)

X-ray, CT, MRI 등 의료 영상에서 종양, 폐렴, 골절 등을 자동 탐지한다. Google DeepMind의 AlphaFold는 단백질 구조 예측 문제를 해결하여 생물학 연구에 혁신을 가져왔다. 유방암 진단에서는 ML 모델이 일부 영상의학과 전문의보다 높은 정확도를 보이는 연구 결과도 발표되었다. 재현율(Recall)이 특히 중요하여, 실제 환자를 놓치지 않는 것이 핵심이다.

4) 음성 인식 (Speech Recognition)

시리(Siri), 구글 어시스턴트, 알렉사(Alexa) 등 음성 비서는 음성을 텍스트로 변환(STT)하고 의도를 파악(NLU)하여 명령을 수행한다. 딥러닝 기반 RNN, Transformer 모델이 핵심이며, 수만 시간의 음성 데이터로 학습한다. 최근에는 OpenAI의 Whisper 모델이 다국어 음성 인식에서 뛰어난 성능을 보인다.

5) 사기 탐지 (Fraud Detection)

금융 기관에서 신용카드 부정 사용, 보험 사기, 자금 세탁 등을 실시간으로 탐지한다. 정상 거래 패턴을 학습한 모델이 이상 거래(anomaly)를 감지하는 방식으로, 비지도학습(이상치 탐지)과 지도학습(분류)이 함께 활용된다. 초당 수천 건의 거래를 실시간으로 분석하며, 매년 수십억 달러 규모의 사기를 방지한다.

1.1.4 머신러닝이 필요한 이유

Mitchell(1997)은 머신러닝이 적용 가능한 상황을 다음과 같이 정리하였다:

1. **복잡한 규칙의 자동 발견**: 사람이 규칙을 일일이 정의하기 어려운 문제 (이미지 인식, 음성 인식)
2. **환경 변화에 대한 적응**: 새로운 데이터가 들어오면 모델을 재학습하여 변화에 대응
3. **대량 데이터에서 패턴 발견**: 빅데이터 시대에 수작업 분석은 불가능

Domingos(2012)는 이에 더해 "더 많은 데이터가 더 똑똑한 알고리즘을 이긴다(More data beats a cleverer algorithm)"는 실무적 교훈을 강조하였다.

1.2 머신러닝의 역사

1.2.1 주요 마일스톤 연대표

연대	주요 사건	의의
1943	McCulloch & Pitts의 인공 뉴런 모델	최초의 수학적 뉴런 모델, 신경망의 이론적 기초
1950	Alan Turing의 "Computing Machinery and Intelligence"	튜링 테스트 제안, "기계가 생각할 수 있는가?"
1957	Frank Rosenblatt의 퍼셉트론(Perceptron)	최초의 학습 가능한 신경망 모델
1959	Arthur Samuel의 체커 프로그램	"Machine Learning" 용어 최초 사용
1967	Cover & Hart의 최근접 이웃법(Nearest Neighbor)	비모수적 분류 알고리즘의 기초
1969	Minsky & Papert의 "Perceptrons"	퍼셉트론의 한계(XOR 문제) 증명, 1차 AI 겨울 촉발
1979	네오코그니트론(Neocognitron), Fukushima	CNN의 원형이 되는 구조 제안
1986	Rumelhart, Hinton, Williams의 역전파(Backpropagation)	다층 신경망 학습 알고리즘, 신경망 부활
1995	Vapnik의 SVM (Support Vector Machine)	통계적 학습 이론 기반의 강력한 분류기
1997	Tom Mitchell의 "Machine Learning" 교재 출판	ML의 형식적 정의(T-P-E) 정립
1997	Wolpert & Macready의 NFL 정리	보편적 최적 알고리즘 불가능성 증명
2001	Hastie, Tibshirani, Friedman의 ESL 출판	통계학 관점의 체계적 ML 교과서
2006	Hinton의 Deep Belief Networks	딥러닝 부활의 신호탄, "딥러닝" 용어 대중화
2012	AlexNet, ImageNet 대회 우승	CNN의 폭발적 성능 향상, 딥러닝 붐 시작
2014	GAN (Generative Adversarial Network), Goodfellow	생성 모델의 혁신
2016	AlphaGo vs 이세돌	강화학습의 가능성을 전 세계에 입증

연대	주요 사건	의의
2017	Transformer 아키텍처 (Attention Is All You Need)	NLP 혁명의 시작, 현대 LLM의 기반
2018~	BERT, GPT 시리즈	대규모 언어 모델(LLM) 시대 개막
2022~	ChatGPT, GPT-4	생성형 AI의 대중화, AGI 논의 활발

1.2.2 AI 겨울과 부활

머신러닝의 역사는 크게 두 번의 **AI 겨울(AI Winter)**과 그 후의 부활로 요약된다:

- **1차 AI 겨울 (1970년대)**: 퍼셉트론의 한계(XOR 문제)가 증명되면서 신경망 연구 침체
- **부활 (1980년대)**: 역전파 알고리즘의 발견으로 다층 신경망 학습이 가능해짐
- **2차 AI 겨울 (1990년대 초)**: 신경망의 학습 어려움(vanishing gradient 등)으로 SVM, 앙상블 등이 주류
- **딥러닝 부활 (2006~2012)**: GPU 컴퓨팅, 빅데이터, 새로운 활성화 함수(ReLU) 등으로 딥러닝이 폭발적 성장

1.3 학습 유형 - 지도/비지도/강화학습

1.3.1 지도학습 (Supervised Learning)

정의: 입력 데이터 X와 그에 대응하는 정답 레이블 y가 함께 주어진 상태에서 학습하는 방법이다. 모델은 입력과 출력 사이의 매핑 함수 $f : X \to y$를 학습한다.

핵심 특징: - 학습 데이터에 정답(Label)이 반드시 포함 - 예측값과 실제값의 차이(오차)를 줄이는 방향으로 학습 - 회귀(Regression)와 분류(Classification)로 세분화

회귀(Regression): 출력값 y가 연속적인 수치인 문제. 예: 주택 가격 예측, 기온 예측

분류(Classification): 출력값 y가 이산적인 카테고리인 문제. 예: 스팸 분류, 질병 진단

산업 사례 3가지:

산업 분야	문제	입력(X)	출력(y)	알고리즘 예시
금융	신용 점수 예측	소득, 부채, 거래 이력	신용 등급 (A~F)	로지스틱 회귀, XGBoost
제조	제품 불량 예측	센서 데이터, 온도, 압력	양품/불량	Random Forest, SVM
마케팅	고객 이탈 예측	사용 패턴, 결제 이력	이탈/유지	Gradient Boosting

1.3.2 비지도학습 (Unsupervised Learning)

정의: 입력 데이터 X만 주어지고 정답 레이블 y가 없는 상태에서 데이터 자체의 구조나 패턴을 발견하는 학습 방법이다.

핵심 특징: - 정답 레이블 없음 (Unlabeled Data) - 데이터의 숨겨진 구조(Hidden Structure)를 발견 - 군집화(Clustering), 차원 축소(Dimensionality Reduction), 이상치 탐지(Anomaly Detection) 등

산업 사례 3가지:

산업 분야	문제	방법	알고리즘 예시
유통	고객 세분화	구매 패턴 유사 고객 그룹화	K-Means, DBSCAN

산업 분야	문제	방법	알고리즘 예시
통신	네트워크 이상 탐지	정상 트래픽 패턴에서 벗어나는 이상 감지	Isolation Forest, Autoencoder
바이오	유전자 군집화	유사 발현 패턴 유전자 그룹화	계층적 군집화, PCA

1.3.3 강화학습 (Reinforcement Learning)

정의: 에이전트(Agent)가 환경(Environment)과 상호작용하면서, 보상(Reward)을 최대화하는 방향으로 행동 전략(Policy)을 학습하는 방법이다.

핵심 구성 요소: - **에이전트(Agent)**: 학습하고 행동하는 주체 - **환경(Environment)**: 에이전트가 상호작용하는 세계 - **상태(State)**: 현재 환경의 상황 - **행동(Action)**: 에이전트가 취할 수 있는 동작 - **보상(Reward)**: 행동에 대한 피드백

```
에이전트 --[행동]--> 환경
환경 --[상태, 보상]--> 에이전트
(이 과정을 반복하며 최적 정책을 학습)
```

산업 사례 3가지:

산업 분야	문제	보상 설계	알고리즘 예시
게임	게임 AI (AlphaGo)	승리 +1, 패배 -1	MCTS + 정책 신경망
로보틱스	로봇 팔 제어	목표물 잡기 성공/실패	PPO, SAC
에너지	데이터센터 냉각 최적화	에너지 절감량	DQN (Google DeepMind)

1.3.4 세 가지 학습 유형 비교

구분	지도학습	비지도학습	강화학습
데이터	입력 + 정답	입력만	상태 + 보상
목표	예측/분류	구조 발견	보상 최대화
피드백	정답 레이블	없음	보상 신호
대표 문제	분류, 회귀	군집화, 차원축소	게임, 로봇 제어
난이도	상대적 쉬움	중간	상대적 어려움

1.4 ML vs DL vs AI 관계

1.4.1 계층 구조

```
인공지능 (Artificial Intelligence)
  +-- 머신러닝 (Machine Learning)
        +-- 딥러닝 (Deep Learning)
```

- **인공지능(AI)**: 인간의 지능을 모방하는 모든 기술의 총칭. 규칙 기반 시스템, 전문가 시스템, 탐색 알고리즘 등을 포함하는 가장 넓은 개념이다.
- **머신러닝(ML)**: AI의 하위 분야로, 데이터 기반 학습에 초점을 맞춘다. 명시적 규칙 대신 데이터에서 패턴을 자동으로 학습한다.

- **딥러닝(DL)**: ML의 하위 분야로, 심층 신경망(Deep Neural Network)을 활용한다. 여러 층(layer)의 비선형 변환을 통해 복잡한 표현을 학습한다.

1.4.2 ML vs DL 상세 비교

특성	전통적 머신러닝	딥러닝
피처 엔지니어링	사람이 직접 설계 (수동)	모델이 자동으로 학습
데이터 요구량	상대적으로 적은 데이터도 가능	대량의 데이터 필요
계산 자원	CPU로 충분한 경우 많음	GPU/TPU 필수적
해석 가능성	상대적으로 높음 (결정 트리 등)	블랙박스 경향
성능 상한	데이터 증가 시 성능 포화	데이터 증가에 따라 지속 향상
적합한 데이터	정형 데이터 (테이블)	비정형 데이터 (이미지, 텍스트, 음성)
대표 알고리즘	SVM, Random Forest, XGBoost	CNN, RNN, Transformer

1.4.3 언제 무엇을 사용할 것인가

Domingos(2012)는 "더 많은 데이터가 더 똑똑한 알고리즘을 이긴다"고 하였지만, 동시에 알고리즘 선택도 중요하다고 강조하였다:

- **정형 데이터 + 소규모~중규모**: 전통적 ML (XGBoost, LightGBM, Random Forest)
- **비정형 데이터 (이미지, 텍스트, 음성)**: 딥러닝 (CNN, Transformer)
- **해석 가능성이 중요한 도메인 (의료, 금융)**: 전통적 ML 또는 설명 가능한 AI(XAI)
- **데이터가 매우 적을 때**: 전통적 ML + 도메인 지식 기반 피처 엔지니어링

1.5 편향-분산 트레이드오프

1.5.1 기본 개념

편향(Bias): 모델의 예측값과 실제 정답 사이의 **체계적인 오차**이다. 모델이 학습 데이터의 패턴을 얼마나 잘 포착하는지를 나타낸다.

- 높은 편향: 모델이 너무 단순하여 복잡한 패턴을 놓침 → 과소적합(Underfitting)
- 낮은 편향: 모델이 데이터의 패턴을 잘 포착함

분산(Variance): 서로 다른 학습 데이터셋으로 모델을 학습시켰을 때 예측값이 **얼마나 크게 변하는지**를 나타낸다.

- 높은 분산: 학습 데이터에 따라 예측이 크게 변함 → 과적합(Overfitting)
- 낮은 분산: 학습 데이터가 바뀌어도 예측이 안정적

1.5.2 수식 유도

실제 데이터가 다음과 같은 모델로 생성된다고 가정한다:

$$y = f(x) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

여기서 $f(x)$는 실제 함수(ground truth), ϵ은 줄일 수 없는 노이즈이다.

학습 데이터셋 $\mathcal{D}$로 학습한 모델의 예측을 $\hat{f}_{\mathcal{D}}(x)$라 하면, 기대 예측 오류(Expected Prediction Error)는:

$$EPE(x) = E_{\mathcal{D}}\left[(y - \hat{f}_{\mathcal{D}}(x))^2\right]$$

이를 전개하면:

$$EPE(x) = E_{\mathcal{D}}\left[(f(x) + \epsilon - \hat{f}_{\mathcal{D}}(x))^2\right]$$

$\bar{f}(x) = E_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]$ (여러 데이터셋에 대한 모델 예측의 기대값)를 도입하고, $\hat{f}_{\mathcal{D}}(x) = \bar{f}(x) + (\hat{f}_{\mathcal{D}}(x) - \bar{f}(x))$로 분해하면:

$$EPE(x) = \underbrace{(f(x) - \bar{f}(x))^2}_{\text{Bias}^2} + \underbrace{E_{\mathcal{D}}\left[(\hat{f}_{\mathcal{D}}(x) - \bar{f}(x))^2\right]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible Error}}$$

따라서 총 오차는 세 가지 성분으로 분해된다:

$$\boxed{EPE = \text{Bias}^2 + \text{Variance} + \sigma^2}$$

각 항의 의미:

항	수식	의미	줄이는 방법
편향^2	$(f(x) - \bar{f}(x))^2$	모델 가정으로 인한 체계적 오차	모델 복잡도 증가
분산	$E[(\hat{f}(x) - \bar{f}(x))^2]$	데이터 변동에 의한 예측 불안정성	모델 복잡도 감소, 앙상블
비가약 오류	σ^2	데이터 자체의 노이즈, 줄일 수 없음	줄일 수 없음

1.5.3 직관적 설명

비유: 과녁 맞추기

과녁의 중심이 실제 정답이라고 하자:

- **낮은 편향 + 낮은 분산**: 화살이 과녁 중심 근처에 밀집 (이상적)
- **높은 편향 + 낮은 분산**: 화살이 한곳에 밀집하지만 중심에서 벗어남 (과소적합)
- **낮은 편향 + 높은 분산**: 화살이 중심 주위에 퍼짐 (과적합)
- **높은 편향 + 높은 분산**: 화살이 중심에서 벗어나면서 퍼짐 (최악)

1.5.4 그래프 해석

```
오차
 ^
 |  \                    /
 |   \   분산(Variance)  /
 |    \               /
 |     \           /
 |      \       /
 |  편향 \   /   <-- 최적 복잡도 (총 오차 최소)
 | (Bias)  \/
 |        / \
 |      /    \___________  총 오차 (Total Error)
 |    /
 |  /
 +-------------------------------> 모델 복잡도
```

- **x축**: 모델 복잡도 (예: 다항식 차수, 트리 깊이, 신경망 층 수)
- **y축**: 오류(Error)

- 모델 복잡도가 증가하면 편향은 감소하고 분산은 증가한다
- **최적의 모델 복잡도**는 편향^{2}과 분산의 합이 최소인 지점에서 달성된다

1.5.5 모델별 편향-분산 특성

모델	편향	분산	특징
선형 회귀	높음	낮음	단순하지만 안정적
KNN (K 큼)	높음	낮음	과도하게 평활화
KNN (K 작음)	낮음	높음	노이즈에 민감
결정 트리 (깊음)	낮음	높음	과적합 경향
랜덤 포레스트	낮음	적절	앙상블로 분산 감소
부스팅	낮음	적절~높음	편향 감소에 강점

ESL(Hastie et al., 2009) Ch2에서는 최소제곱법(높은 편향, 낮은 분산)과 k-NN(낮은 편향, 높은 분산)의 대비를 통해 이 트레이드오프를 직관적으로 설명한다.

1.6 No Free Lunch 정리

1.6.1 정리의 내용

Wolpert & Macready(1997)가 증명한 **No Free Lunch(NFL) 정리**는 머신러닝과 최적화 분야에서 가장 근본적인 이론적 결과 중 하나이다.

> **핵심 주장**: 모든 가능한 문제(목적 함수)에 대해 평균을 내면, 어떤 두 최적화(또는 학습) 알고리즘이든 동일한 성능을 보인다.

1.6.2 수학적 정의

최적화 문제 설정: - $\mathcal{X}$: 후보 해(candidate solution)의 집합 - $\mathcal{Y}$: 비용 값(cost value)의 집합 - $f : \mathcal{X} \to \mathcal{Y}$: 목적 함수 - a: 최적화 알고리즘

NFL 정리 (형식적 표현):

모든 가능한 함수 f에 대해 합산하면:

$$\sum_f P(\mathbf{d}_m^y \mid f, m, a_1) = \sum_f P(\mathbf{d}_m^y \mid f, m, a_2)$$

여기서 $\mathbf{d}_m^y$는 알고리즘이 m번 평가한 후의 성능 벡터이다. 이 등식은 모든 알고리즘 쌍 a_1, a_2에 대해 성립한다.

직관적 의미: - 알고리즘 a_1이 특정 문제 집합에서 a_2보다 우수하면, 반드시 다른 문제 집합에서는 a_2가 a_1보다 우수 - 모든 함수에 대한 평균 성능은 모든 알고리즘이 동일 - 무작위 탐색(random search)도 정교한 알고리즘과 평균 성능이 동일

1.6.3 핵심 가정과 한계

가정	의미	현실적 해석
모든 함수에 대한 균일 분포	가능한 모든 f를 동등하게 고려	현실 문제는 특정 구조를 가짐
이전 평가 재방문 없음	같은 점을 두 번 평가하지 않음	대부분의 알고리즘이 만족
비용 함수의 결정적 특성	f가 deterministic	확률적 문제에는 별도 분석 필요

NFL 정리는 "모든 알고리즘이 동등하다"로 오해되지만, 이는 정확하지 않다. 현실 세계의 문제는 "모든 가능한 함수의 균일 분포"와 매우 다르며, 현실 문제는 구조(structure)를 가지고 있다.

1.6.4 실무적 시사점

1. **"최고의 알고리즘"은 없다**: 항상 문제에 맞는 알고리즘을 선택해야 함
2. **사전 지식의 활용**: 문제의 구조에 대한 지식(귀납적 편향)이 알고리즘 선택의 핵심
3. **다양한 알고리즘 시도**: 하나의 알고리즘에 의존하지 말고 여러 알고리즘을 비교
4. **벤치마크의 의미**: 특정 벤치마크에서의 우수한 성능이 모든 문제에서의 우수성을 의미하지 않음
5. **AutoML의 이론적 근거**: 자동 알고리즘 선택/하이퍼파라미터 튜닝의 당위성

NFL 정리는 "어떤 알고리즘을 써야 하나요?"라는 가장 흔한 질문에 대한 근본적 답을 제공한다: **"문제에 따라 다르다. 따라서 다양한 알고리즘을 배워야 한다."**

1.7 모델 평가 - 교차검증 전략 비교

1.7.1 왜 데이터를 나누는가?

모델의 **일반화 성능(Generalization Performance)**을 올바르게 평가하려면, 학습에 사용한 데이터와 평가에 사용하는 데이터를 반드시 분리해야 한다. 학습 데이터로 평가하면 과적합 여부를 알 수 없다.

1.7.2 Hold-Out

가장 단순한 방법으로, 데이터를 한 번만 학습/테스트로 분할한다 (예: 80:20).

- **장점**: 계산 비용이 가장 낮음, 구현이 간단
- **단점**: 분할에 따라 결과가 크게 달라짐 (높은 분산), 데이터를 비효율적으로 사용
- **사용 시기**: 데이터가 매우 많을 때, 빠른 프로토타이핑

1.7.3 K-Fold 교차검증

데이터를 K개의 폴드(Fold)로 나누어 K번 평가한다.

```
Fold 1: [검증] [학습] [학습] [학습] [학습]  -> 성능1
Fold 2: [학습] [검증] [학습] [학습] [학습]  -> 성능2
Fold 3: [학습] [학습] [검증] [학습] [학습]  -> 성능3
Fold 4: [학습] [학습] [학습] [검증] [학습]  -> 성능4
Fold 5: [학습] [학습] [학습] [학습] [검증]  -> 성능5

최종 성능 = (성능1 + 성능2 + ... + 성능K) / K
```

- **장점**: 모든 데이터를 학습과 평가에 사용, Hold-Out보다 낮은 분산
- **단점**: 클래스 불균형 시 폴드별 클래스 비율이 다를 수 있음
- **사용 시기**: 일반적인 모델 평가 ($K = 5$ 또는 $K = 10$이 표준)

1.7.4 Stratified K-Fold

K-Fold와 동일하지만, 각 폴드에서 **클래스 비율을 원본과 동일하게 유지**한다.

- **장점**: 클래스 불균형 데이터에서 안정적, 가장 널리 권장됨
- **단점**: 회귀 문제에는 직접 적용 불가 (연속값이므로 층화 불가)
- **사용 시기**: 분류 문제, 특히 클래스 불균형 시 (sklearn의 기본 CV 전략)

1.7.5 Leave-One-Out (LOO)

한 번에 하나의 샘플만 테스트로 사용하는 극단적인 K-Fold (K=N).

- **장점**: 편향이 가장 낮음 (거의 전체 데이터로 학습)

- **단점**: 계산 비용이 매우 높음 (N번 학습), 분산이 높을 수 있음 (학습셋 간 중복이 매우 큼)
- **사용 시기**: 데이터가 매우 적을 때 (N < 50)

1.7.6 네 가지 전략 비교

전략	학습 데이터 비율	평가 횟수	편향	분산	계산 비용
Hold-Out	고정 (보통 80%)	1회	높음	높음	매우 낮음
5-Fold	80% (각 반복)	5회	중간	중간	중간
10-Fold	90% (각 반복)	10회	낮음	중간~높음	중간~높음
LOO	$(N-1)/N$	N회	가장 낮음	높음	매우 높음

1.7.7 K값에 따른 편향-분산 트레이드오프

K-Fold에서 K값의 선택은 그 자체로 편향-분산 트레이드오프를 포함한다:

- **K가 작을 때 (예: $K = 2$)**: 학습 데이터가 50%뿐이므로 높은 편향, 하지만 폴드 간 독립성이 높아 낮은 분산
- **K가 클 때 (예: $K = N$, LOO)**: 거의 전체 데이터로 학습하므로 낮은 편향, 하지만 폴드 간 학습셋이 거의 동일하여 높은 분산
- **실무적 최적점**: $K = 5 \sim 10$이 가장 일반적으로 권장됨

1.7.8 데이터 크기별 권장 전략

데이터 크기	권장 전략	이유
대용량 (> 10,000)	Hold-Out 또는 3-Fold	충분한 데이터, 계산 효율성
중간 (100 ~ 10,000)	Stratified 5-Fold 또는 10-Fold	안정적 추정, 적절한 계산 비용
소용량 (< 100)	LOO 또는 Repeated K-Fold	모든 데이터를 최대한 활용

1.8 ML 파이프라인과 데이터 품질

1.8.1 머신러닝 파이프라인

머신러닝 프로젝트는 일반적으로 다음과 같은 단계적 워크플로우를 따른다:

```
[1. 문제 정의] -> [2. 데이터 수집] -> [3. 데이터 전처리] -> [4. EDA]
    -> [5. 모델링] -> [6. 평가] -> [7. 하이퍼파라미터 튜닝] -> [8. 배포]
```

단계	설명	핵심 활동
문제 정의	비즈니스 문제를 ML 문제로 변환	회귀/분류/군집화 결정, 성공 기준 설정
데이터 수집	필요한 데이터 확보	DB, API, 크롤링, 공개 데이터셋
데이터 전처리	원시 데이터를 모델용으로 가공	결측치, 이상치, 인코딩, 스케일링
EDA	데이터 특성 파악	분포, 상관관계, 시각화
모델링	알고리즘 선택 및 학습	여러 모델 비교 실험
평가	일반화 성능 측정	교차검증, 테스트셋 평가
HP 튜닝	하이퍼파라미터 최적화	Grid Search, Random Search, Bayesian
배포	실서비스 적용	REST API, 모니터링, 재학습

1.8.2 데이터 품질의 중요성

"Garbage In, Garbage Out": 나쁜 데이터로는 좋은 모델을 만들 수 없다.

Domingos(2012)는 "More data beats a cleverer algorithm"이라고 하였다. 이는 데이터의 양이 중요하다는 의미이지만, 동시에 데이터의 **질**도 핵심적이다.

데이터 전처리 주요 단계:

단계	설명	방법
결측치 처리	누락된 데이터 처리	삭제, 평균/중앙값 대체, 보간
이상치 처리	비정상적 극단값 처리	IQR, Z-score 기반 탐지
인코딩	범주형 변수를 수치로 변환	원핫 인코딩, 레이블 인코딩
스케일링	변수의 크기/범위 통일	StandardScaler, MinMaxScaler
피처 엔지니어링	새로운 특성 생성/선택	도메인 지식 활용, 특성 조합

1.8.3 피처 엔지니어링의 중요성

Domingos(2012)의 교훈 7: **"Feature Engineering Is the Key"**

많은 ML 프로젝트에서 알고리즘 선택보다 피처 설계가 더 큰 영향을 미친다. 도메인 지식을 활용한 적절한 피처 구성이 모델 성능을 결정짓는 핵심 요소이다.

예를 들어, 주택 가격 예측에서: - 단순 피처: 면적, 방 수, 위치 - 엔지니어링된 피처: 방당 면적, 지하철역까지 거리, 학군 등급

1.8.4 데이터 누출 (Data Leakage) 방지

데이터 누출은 학습 과정에서 테스트 데이터의 정보가 모델에 유입되는 현상으로, 과도하게 낙관적인 성능 추정을 초래한다.

주요 방지 원칙: - 스케일링은 학습셋에서 `fit_transform`, 테스트셋에서는 `transform`만 수행 - 교차검증 루프 안에서 전처리를 수행 (sklearn의 `Pipeline` 활용) - 시계열 데이터에서는 미래 데이터를 학습에 사용하지 않음

1.9 핵심 논문 리뷰

1.9.1 Domingos (2012) - "A Few Useful Things to Know About Machine Learning"

항목	내용
저자	Pedro Domingos
출처	Communications of the ACM, Vol. 55, No. 10, pp. 78-87
핵심	ML 실무에서 반드시 알아야 할 12가지 교훈

12가지 핵심 교훈 요약:

번호	교훈	핵심 내용
1	Representation + Evaluation + Optimization	모든 학습 알고리즘의 세 구성 요소
2	It' s Generalization That Counts	일반화 성능이 핵심
3	Data Alone Is Not Enough	사전 가정(inductive bias)이 필요

번호	교훈	핵심 내용
4	Overfitting Has Many Faces	과적합은 다양한 형태로 나타남
5	Intuition Fails in High Dimensions	차원의 저주
6	Theoretical Guarantees Are Not What They Seem	이론적 보장의 한계
7	Feature Engineering Is the Key	피처 설계가 핵심
8	More Data Beats a Cleverer Algorithm	데이터 양의 중요성
9	Learn Many Models, Not Just One	앙상블의 우수성
10	Simplicity Does Not Imply Accuracy	단순성 $\neq$ 정확도
11	Representable Does Not Imply Learnable	표현 가능 $\neq$ 학습 가능
12	Correlation Does Not Imply Causation	상관관계 $\neq$ 인과관계

1.9.2 Mitchell (1997) - "Machine Learning" Ch1-2

항목	내용
저자	Tom M. Mitchell
출판	McGraw-Hill, 1997
핵심	ML의 형식적 정의(T-P-E), 개념 학습, 귀납적 편향

주요 기여: - **ML의 형식적 정의**: "프로그램이 경험 E로부터 작업 T에 대해 학습하며, 성능 P가 E를 통해 향상되면 학습한다고 한다" - **개념 학습 프레임워크**: 가설 공간($\mathcal{H}$), 버전 스페이스, Find-S 알고리즘 - **귀납적 편향(Inductive Bias)**: 모든 학습 알고리즘은 새로운 데이터에 일반화하기 위해 사전 가정을 필요로 함. 이는 NFL 정리와 직접 연결됨

1.9.3 Wolpert & Macready (1997) - "No Free Lunch Theorems for Optimization"

항목	내용
저자	David H. Wolpert, William G. Macready
출처	IEEE Transactions on Evolutionary Computation, Vol. 1, No. 1
핵심	보편적 최적 알고리즘의 불가능성 증명

핵심 결과: 모든 가능한 함수 f에 대해 평균하면, 어떤 두 알고리즘의 성능도 동일하다. 따라서 "이 특정 문제에 어떤 알고리즘이 가장 적합한가?"가 올바른 질문이다.

1.9.4 Vapnik (1995) - "The Nature of Statistical Learning Theory"

항목	내용
저자	Vladimir N. Vapnik
출판	Springer-Verlag, 1995

항목	내용
핵심	통계적 학습 이론, VC 차원, SRM, SVM

주요 개념:

개념	수식/설명
경험적 위험	$R_{\text{emp}}(f) = \frac{1}{n}\sum_{i=1}^{n} L(f(x_i), y_i)$
실제 위험	$R(f) = \int L(f(x), y)\, dP(x, y)$
VC 차원	가설 공간이 shatter할 수 있는 최대 점 수
구조적 위험 최소화	$R(f) \leq R_{\text{emp}}(f) + \Phi(\text{VCdim}, n)$

Vapnik의 핵심 물음: "유한한 학습 데이터로부터 학습한 모델이 새로운 데이터에서도 잘 작동할 조건은 무엇인가?" 이에 대한 답으로 VC 차원과 구조적 위험 최소화 원리를 제시하였다.

1.9.5 Hastie, Tibshirani, Friedman (2009) - "The Elements of Statistical Learning" Ch1-2

항목	내용
저자	Trevor Hastie, Robert Tibshirani, Jerome Friedman
출판	Springer, 2009 (2판)
핵심	편향-분산 분해, 통계적 결정 이론, 최소제곱 vs k-NN

주요 기여: - **기대 예측 오류(EPE)**: $EPE(f) = E[(Y - f(X))^2]$ - **최적 예측기**: 회귀에서는 조건부 기대값 $f^*(x) = E[Y|X = x]$, 분류에서는 베이즈 분류기 - **편향-분산 분해**: $EPE = \text{Bias}^2 + \text{Variance} + \sigma^2$ - **최소제곱법 vs k-NN 비교**를 통한 편향-분산 트레이드오프의 직관적 설명:

특성	최소제곱법 (Linear)	k-NN
가정	선형 관계 (강한 가정)	국소적 부드러움 (약한 가정)
편향	높음	낮음
분산	낮음	높음
적합한 상황	선형 관계, 적은 데이터	비선형 관계, 많은 데이터

1.10 실습: 편향-분산 트레이드오프 구현

1.10.1 실습 목표

- 다양한 복잡도의 다항 회귀 모델을 학습하여 편향^2, 분산, 총 오류를 시각화한다
- 과소적합/과적합 영역을 직관적으로 확인한다
- 편향-분산 분해 공식을 실험적으로 검증한다

1.10.2 전체 코드

```
# ============================================================
# 01_bias_variance_tradeoff.py
# 편향-분산 트레이드오프 (Bias-Variance Tradeoff) 시각화
#
# 참고 논문: Pedro Domingos (2012) "A Few Useful Things to
#            Know About Machine Learning"
```

```
#
# 이 코드는 다양한 복잡도의 다항 회귀 모델을 학습하여
# 편향(Bias), 분산(Variance), 총 오류(Total Error)를
# 시각화하고, 과소적합/과적합 영역을 보여준다.
# ============================================================

import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error

# --- 한글 폰트 설정 ---
plt.rcParams['font.family'] = 'Malgun Gothic'
plt.rcParams['axes.unicode_minus'] = False

# --- 시드 고정 (재현성) ---
np.random.seed(42)


# ============================================================
# 1. 실제 함수 (True Function) 정의
# ============================================================
def true_function(x):
    """
    실제 데이터 생성 함수 (ground truth).
    비선형 관계를 가진 함수로, 다항 회귀의 적합성을 테스트하기에 적합하다.
    """
    return np.sin(1.5 * np.pi * x)


# ============================================================
# 2. 합성 데이터 생성 함수
# ============================================================
def generate_data(n_samples=30, noise_std=0.3):
    """
    합성 데이터 생성.

    Parameters
    ----------
    n_samples : int
        생성할 데이터 포인트 수
    noise_std : float
        가우시안 노이즈의 표준편차

    Returns
    -------
    X : ndarray, shape (n_samples,)
        입력 변수
    y : ndarray, shape (n_samples,)
        출력 변수 (실제 함수 + 노이즈)
    """
```

```
    X = np.sort(np.random.uniform(0, 1, n_samples))
    y = true_function(X) + np.random.normal(0, noise_std, n_samples)
    return X, y


# ============================================================
# 3. 편향-분산 계산
# ============================================================
def compute_bias_variance(degrees, n_datasets=200, n_samples=30,
                          noise_std=0.3, n_test=100):
    """
    여러 다항식 차수에 대해 편향^2, 분산, 총 오류를 계산한다.

    원리:
    - 동일한 분포에서 n_datasets개의 학습 데이터셋을 생성
    - 각 데이터셋으로 모델을 학습
    - 고정된 테스트 포인트에서 예측값의 편향과 분산을 계산

    편향-분산 분해 (Bias-Variance Decomposition):
        E[(y - f_hat(x))^2] = Bias^2(f_hat(x)) + Var(f_hat(x)) + sigma^2

    Parameters
    ----------
    degrees : list of int
        테스트할 다항식 차수 리스트
    n_datasets : int
        반복 생성할 데이터셋 수 (평균을 내기 위함)
    n_samples : int
        각 데이터셋의 샘플 수
    noise_std : float
        노이즈 표준편차
    n_test : int
        테스트 포인트 수

    Returns
    -------
    bias_squared : dict
        각 차수별 평균 편향^2
    variance : dict
        각 차수별 평균 분산
    total_error : dict
        각 차수별 평균 총 오류 (MSE)
    """
    # 고정된 테스트 포인트
    X_test = np.linspace(0, 1, n_test)
    y_true = true_function(X_test)
    noise_var = noise_std ** 2  # 비가약 오류 (irreducible error)

    bias_squared = {}
    variance = {}
    total_error = {}

    for degree in degrees:
```

```
        # 각 데이터셋에서의 예측값을 저장할 배열
        # shape: (n_datasets, n_test)
        predictions = np.zeros((n_datasets, n_test))

        for i in range(n_datasets):
            # 새로운 학습 데이터 생성
            X_train, y_train = generate_data(n_samples, noise_std)

            # 다항 회귀 모델 학습
            model = make_pipeline(
                PolynomialFeatures(degree, include_bias=True),
                LinearRegression()
            )
            model.fit(X_train.reshape(-1, 1), y_train)

            # 테스트 포인트에서 예측
            predictions[i, :] = model.predict(X_test.reshape(-1, 1))

        # 편향^2 계산: (평균 예측 - 실제값)^2의 평균
        mean_prediction = predictions.mean(axis=0)
        bias_sq = np.mean((mean_prediction - y_true) ** 2)

        # 분산 계산: 예측값의 분산의 평균
        var = np.mean(predictions.var(axis=0))

        # 총 오류 (MSE): 편향^2 + 분산 + 노이즈
        total = bias_sq + var + noise_var

        bias_squared[degree] = bias_sq
        variance[degree] = var
        total_error[degree] = total

        print(f"  차수 {degree:2d}: Bias^2={bias_sq:.4f}, "
              f"Var={var:.4f}, Noise={noise_var:.4f}, "
              f"Total={total:.4f}")

    return bias_squared, variance, total_error


# ============================================================
# 4. 시각화 1: 다양한 복잡도의 모델 적합 결과
# ============================================================
def plot_model_fits(degrees):
    """
    다양한 차수의 다항 회귀 적합 결과를 시각화한다.
    과소적합(저차수)과 과적합(고차수)을 직관적으로 보여준다.
    """
    X_train, y_train = generate_data(n_samples=30, noise_std=0.3)
    X_plot = np.linspace(0, 1, 200)

    fig, axes = plt.subplots(2, 3, figsize=(18, 11))
    axes = axes.flatten()
```

```
    for idx, degree in enumerate(degrees):
        ax = axes[idx]

        # 모델 학습
        model = make_pipeline(
            PolynomialFeatures(degree, include_bias=True),
            LinearRegression()
        )
        model.fit(X_train.reshape(-1, 1), y_train)
        y_pred = model.predict(X_plot.reshape(-1, 1))

        # 학습 MSE 계산
        train_pred = model.predict(X_train.reshape(-1, 1))
        train_mse = mean_squared_error(y_train, train_pred)

        # 시각화
        ax.scatter(X_train, y_train, color='steelblue', s=30,
                   alpha=0.7, label='학습 데이터', zorder=3)
        ax.plot(X_plot, true_function(X_plot), 'g--', linewidth=2,
                label='실제 함수 $f(x)$', alpha=0.7)
        ax.plot(X_plot, y_pred, 'r-', linewidth=2,
                label=f'다항식 (차수={degree})')

        # 상태 표시
        if degree <= 2:
            status = "과소적합 (Underfitting)"
        elif degree <= 5:
            status = "적절한 적합 (Good Fit)"
        else:
            status = "과적합 (Overfitting)"

        ax.set_title(f'다항식 차수 = {degree}\n{status}\n'
                     f'학습 MSE = {train_mse:.4f}', fontsize=11)
        ax.set_xlabel('x')
        ax.set_ylabel('y')
        ax.set_ylim(-2, 2)
        ax.legend(fontsize=8, loc='upper right')
        ax.grid(True, alpha=0.3)

    plt.suptitle('다항 회귀의 모델 복잡도에 따른 적합 결과',
                 fontsize=16, fontweight='bold', y=1.01)
    plt.tight_layout()
    plt.savefig('01_model_fits.png', dpi=150, bbox_inches='tight')
    plt.show()
    print("[저장 완료] 01_model_fits.png")


# ============================================================
# 5. 시각화 2: 편향-분산 트레이드오프 곡선
# ============================================================
def plot_bias_variance_tradeoff(degrees, bias_sq, var, total):
    """
    편향^2, 분산, 총 오류를 모델 복잡도(다항식 차수)에 따라 플롯한다.
```

```
    """
    deg_list = sorted(degrees)
    bias_vals = [bias_sq[d] for d in deg_list]
    var_vals = [var[d] for d in deg_list]
    total_vals = [total[d] for d in deg_list]
    noise_val = 0.3 ** 2

    fig, ax = plt.subplots(figsize=(10, 7))

    ax.plot(deg_list, bias_vals, 'b-o', linewidth=2, markersize=8,
            label='편향^2 (Bias^2)', zorder=3)
    ax.plot(deg_list, var_vals, 'r-s', linewidth=2, markersize=8,
            label='분산 (Variance)', zorder=3)
    ax.plot(deg_list, total_vals, 'k-^', linewidth=2.5, markersize=9,
            label='총 오류 (Total Error)', zorder=3)
    ax.axhline(y=noise_val, color='gray', linestyle='--', linewidth=1.5,
               label=f'비가약 오류 (sigma^2={noise_val:.2f})', alpha=0.7)

    # 최적 지점 표시
    optimal_degree = deg_list[np.argmin(total_vals)]
    optimal_error = min(total_vals)
    ax.axvline(x=optimal_degree, color='green', linestyle=':', linewidth=2,
               alpha=0.7, label=f'최적 복잡도 (차수={optimal_degree})')
    ax.scatter([optimal_degree], [optimal_error], color='green', s=200,
               zorder=5, marker='*', edgecolor='black', linewidth=1)

    ax.set_xlabel('모델 복잡도 (다항식 차수)', fontsize=13)
    ax.set_ylabel('오류 (Error)', fontsize=13)
    ax.set_title('편향-분산 트레이드오프 (Bias-Variance Tradeoff)',
                 fontsize=14, fontweight='bold')
    ax.legend(fontsize=10, loc='upper center',
              bbox_to_anchor=(0.5, -0.12), ncol=3)
    ax.set_xticks(deg_list)
    ax.grid(True, alpha=0.3)
    ax.set_ylim(bottom=0)

    plt.tight_layout()
    plt.savefig('01_bias_variance_tradeoff.png', dpi=150, bbox_inches='tight')
    plt.show()
    print("[저장 완료] 01_bias_variance_tradeoff.png")


# ============================================================
# 6. 시각화 3: 여러 데이터셋에서의 예측 불안정성 비교
# ============================================================
def plot_prediction_variability():
    """
    저복잡도 vs 고복잡도 모델에서
    여러 데이터셋에 대한 예측의 변동(분산)을 시각화한다.
    """
    fig, axes = plt.subplots(1, 3, figsize=(18, 5.5))
    X_plot = np.linspace(0, 1, 200)
```

```
    model_configs = [
        (1, '차수 1 (높은 편향, 낮은 분산)'),
        (5, '차수 5 (적절한 균형)'),
        (15, '차수 15 (낮은 편향, 높은 분산)')
    ]

    for ax, (degree, title) in zip(axes, model_configs):
        for i in range(30):
            X_train, y_train = generate_data(n_samples=30, noise_std=0.3)
            model = make_pipeline(
                PolynomialFeatures(degree, include_bias=True),
                LinearRegression()
            )
            model.fit(X_train.reshape(-1, 1), y_train)
            y_pred = model.predict(X_plot.reshape(-1, 1))
            y_pred = np.clip(y_pred, -3, 3)
            ax.plot(X_plot, y_pred, 'r-', alpha=0.15, linewidth=0.8)

        ax.plot(X_plot, true_function(X_plot), 'g-', linewidth=3,
                label='실제 함수', zorder=5)
        ax.set_title(title, fontsize=12, fontweight='bold')
        ax.set_xlabel('x')
        ax.set_ylabel('y')
        ax.set_ylim(-3, 3)
        ax.legend(fontsize=10)
        ax.grid(True, alpha=0.3)

    plt.suptitle('모델 복잡도에 따른 예측의 변동성',
                 fontsize=13, fontweight='bold', y=1.02)
    plt.tight_layout()
    plt.savefig('01_prediction_variability.png', dpi=150, bbox_inches='tight')
    plt.show()
    print("[저장 완료] 01_prediction_variability.png")


# ============================================================
# 메인 실행
# ============================================================
if __name__ == "__main__":
    print("=" * 60)
    print("편향-분산 트레이드오프 (Bias-Variance Tradeoff) 시뮬레이션")
    print("참고: Domingos (2012)")
    print("=" * 60)

    degrees = [1, 3, 5, 10, 15]

    print("\n[1/3] 다양한 복잡도의 모델 적합 결과 시각화...")
    plot_model_fits(degrees)

    fine_degrees = list(range(1, 16))
    print(f"\n[2/3] 편향-분산 계산 (차수 1~15, 200개 데이터셋 반복)...")
    bias_sq, var, total = compute_bias_variance(
        fine_degrees, n_datasets=200, n_samples=30, noise_std=0.3
```

```
    )

    print("\n편향-분산 트레이드오프 곡선 시각화...")
    plot_bias_variance_tradeoff(fine_degrees, bias_sq, var, total)

    print("\n[3/3] 예측 변동성 비교 시각화...")
    plot_prediction_variability()

    optimal_degree = min(total, key=total.get)
    print(f"\n최적 다항식 차수: {optimal_degree}")
    print(f"  - Bias^2: {bias_sq[optimal_degree]:.4f}")
    print(f"  - Variance: {var[optimal_degree]:.4f}")
    print(f"  - Total Error: {total[optimal_degree]:.4f}")
```

1.10.3 코드 해설

1. **실제 함수 정의**: $f(x) = \sin(1.5\pi x)$를 ground truth로 사용
2. **합성 데이터 생성**: 실제 함수에 가우시안 노이즈($\sigma = 0.3$)를 추가
3. **편향-분산 계산**: 200개의 서로 다른 데이터셋으로 모델을 반복 학습하여, 예측의 평균(편향)과 변동(분산)을 실험적으로 계산
4. **시각화**: 모델 적합 결과, 편향-분산 트레이드오프 곡선, 예측 변동성을 세 가지 그래프로 표현

1.11 실습: NFL 정리 데모

1.11.1 실습 목표

- 4가지 서로 다른 구조의 데이터셋에 5가지 알고리즘을 적용한다
- 어떤 단일 알고리즘도 모든 데이터셋에서 최상이 아님을 실험적으로 확인한다
- NFL 정리의 실무적 의미를 체감한다

1.11.2 전체 코드

```
# ============================================================
# 02_no_free_lunch_demo.py
# No Free Lunch 정리 데모
#
# 참고 논문: Wolpert & Macready (1997) "No Free Lunch Theorems
#            for Optimization"
#
# 이 코드는 다양한 데이터셋에 여러 알고리즘을 적용하여,
# 어떤 단일 알고리즘도 모든 데이터셋에서 최상이 아님을 보여준다.
# ============================================================

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

from sklearn.datasets import make_moons, make_circles, make_classification
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
```

```python
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# --- 한글 폰트 설정 ---
plt.rcParams['font.family'] = 'Malgun Gothic'
plt.rcParams['axes.unicode_minus'] = False

# --- 시드 고정 ---
np.random.seed(42)


# ============================================================
# 1. 다양한 데이터셋 생성
# ============================================================
def create_datasets():
    """
    4가지 서로 다른 구조를 가진 데이터셋을 생성한다.
    각 데이터셋은 서로 다른 알고리즘에 유리한 구조를 가진다.
    """
    n_samples = 300

    # 데이터셋 1: 선형 분리 가능 -> 선형 모델에 유리
    X_linear, y_linear = make_classification(
        n_samples=n_samples, n_features=2, n_redundant=0,
        n_informative=2, n_clusters_per_class=1,
        class_sep=2.0, random_state=42
    )

    # 데이터셋 2: 원형 -> RBF 커널 SVM에 유리
    X_circles, y_circles = make_circles(
        n_samples=n_samples, noise=0.1, factor=0.5, random_state=42
    )

    # 데이터셋 3: XOR 패턴 -> 결정 트리에 유리
    X_xor = np.random.randn(n_samples, 2)
    y_xor = np.logical_xor(X_xor[:, 0] > 0, X_xor[:, 1] > 0).astype(int)

    # 데이터셋 4: 반달 -> k-NN / 비선형 모델에 유리
    X_moons, y_moons = make_moons(
        n_samples=n_samples, noise=0.2, random_state=42
    )

    datasets = [
        (X_linear, y_linear, "선형 분리 (Linear)",
         "선형 결정 경계로 분리 가능한 데이터"),
        (X_circles, y_circles, "원형 (Circles)",
         "동심원 구조의 비선형 데이터"),
        (X_xor, y_xor, "XOR 패턴",
         "XOR 논리 연산 구조의 데이터"),
        (X_moons, y_moons, "반달 (Moons)",
         "반달 형태의 비선형 데이터"),
    ]
```

```
    return datasets


# ============================================================
# 2. 알고리즘 정의
# ============================================================
def create_classifiers():
    """
    5가지 대표적인 분류 알고리즘을 정의한다.
    각 알고리즘은 서로 다른 귀납적 편향(inductive bias)을 가진다.
    """
    classifiers = [
        (LogisticRegression(random_state=42, max_iter=1000),
         "로지스틱 회귀 (Linear)", "선형 결정 경계 가정"),
        (KNeighborsClassifier(n_neighbors=5),
         "k-NN (k=5)", "국소적 유사성 가정"),
        (SVC(kernel='rbf', gamma='auto', random_state=42),
         "SVM (RBF Kernel)", "마진 최대화 + 비선형 커널"),
        (DecisionTreeClassifier(max_depth=10, random_state=42),
         "결정 트리", "축 정렬 분할 가정"),
        (RandomForestClassifier(n_estimators=100, random_state=42),
         "랜덤 포레스트", "앙상블 + 축 정렬 분할"),
    ]
    return classifiers


# ============================================================
# 3. 교차 검증으로 성능 평가
# ============================================================
def evaluate_all(datasets, classifiers):
    """
    모든 데이터셋-알고리즘 조합에 대해 교차 검증 성능을 계산한다.
    """
    results = {}
    n_datasets = len(datasets)
    n_classifiers = len(classifiers)
    scores_matrix = np.zeros((n_datasets, n_classifiers))

    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

    for i, (X, y, ds_name, _) in enumerate(datasets):
        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)

        results[ds_name] = {}
        for j, (clf, clf_name, _) in enumerate(classifiers):
            scores = cross_val_score(clf, X_scaled, y, cv=cv,
                                     scoring='accuracy')
            mean_score = scores.mean()
            std_score = scores.std()
            results[ds_name][clf_name] = (mean_score, std_score)
            scores_matrix[i, j] = mean_score
```

```
    return results, scores_matrix


# ============================================================
# 4. 시각화: 성능 비교 히트맵
# ============================================================
def plot_performance_heatmap(datasets, classifiers, scores_matrix):
    """
    데이터셋 x 알고리즘의 성능 히트맵을 그린다.
    """
    ds_names = [ds[2] for ds in datasets]
    clf_names = [clf[1] for clf in classifiers]

    fig, ax = plt.subplots(figsize=(12, 6))

    im = ax.imshow(scores_matrix, cmap='YlOrRd', aspect='auto',
                   vmin=0.5, vmax=1.0)

    ax.set_xticks(range(len(clf_names)))
    ax.set_xticklabels(clf_names, fontsize=10, rotation=15, ha='right')
    ax.set_yticks(range(len(ds_names)))
    ax.set_yticklabels(ds_names, fontsize=11)

    for i in range(len(ds_names)):
        best_j = np.argmax(scores_matrix[i])
        for j in range(len(clf_names)):
            val = scores_matrix[i, j]
            color = 'white' if val > 0.85 else 'black'
            weight = 'bold' if j == best_j else 'normal'
            text = f'{val:.1%}'
            if j == best_j:
                text += '\n(Best)'
            ax.text(j, i, text, ha='center', va='center',
                    color=color, fontsize=10, fontweight=weight)

    plt.colorbar(im, ax=ax, label='교차 검증 정확도', shrink=0.8)
    ax.set_title('No Free Lunch 정리: 데이터셋별 알고리즘 성능 비교\n'
                 '(5-Fold 교차 검증 정확도)',
                 fontsize=13, fontweight='bold')
    plt.tight_layout()
    plt.savefig('02_performance_heatmap.png', dpi=150, bbox_inches='tight')
    plt.show()


# ============================================================
# 5. 결과 표 출력
# ============================================================
def print_comparison_table(datasets, classifiers, results, scores_matrix):
    """콘솔에 비교 결과 표를 출력한다."""
    ds_names = [ds[2] for ds in datasets]
    clf_names = [clf[1] for clf in classifiers]
```

```
    print("\n" + "=" * 90)
    print("No Free Lunch 정리 검증: 데이터셋별 알고리즘 성능 비교표")
    print("=" * 90)

    for i, ds_name in enumerate(ds_names):
        best_j = np.argmax(scores_matrix[i])
        print(f"\n{ds_name}:")
        for j, clf_name in enumerate(clf_names):
            score = scores_matrix[i, j]
            marker = " <-- Best" if j == best_j else ""
            print(f"  {clf_name:<25}: {score:.3f}{marker}")

    print(f"\n핵심 결론 (NFL 정리):")
    print(f"  -> 모든 데이터셋에서 1위인 단일 알고리즘은 존재하지 않는다!")


# ============================================================
# 메인 실행
# ============================================================
if __name__ == "__main__":
    print("=" * 60)
    print("No Free Lunch 정리 데모")
    print("참고: Wolpert & Macready (1997)")
    print("=" * 60)

    datasets = create_datasets()
    classifiers = create_classifiers()

    print(f"\n사용 데이터셋 ({len(datasets)}개):")
    for _, _, name, desc in datasets:
        print(f"  - {name}: {desc}")

    print(f"\n사용 알고리즘 ({len(classifiers)}개):")
    for _, name, bias in classifiers:
        print(f"  - {name}: {bias}")

    print(f"\n[1/2] 교차 검증 성능 평가 중...")
    results, scores_matrix = evaluate_all(datasets, classifiers)

    print_comparison_table(datasets, classifiers, results, scores_matrix)

    print(f"\n[2/2] 성능 히트맵 시각화 중...")
    plot_performance_heatmap(datasets, classifiers, scores_matrix)

    print("\n최종 요약: NFL 정리의 시사점")
    print("  1. 모든 문제에 대해 최상인 알고리즘은 존재하지 않는다.")
    print("  2. 문제의 특성에 맞는 알고리즘을 선택하는 것이 핵심이다.")
    print("  3. 항상 여러 알고리즘을 시도하고 비교할 것.")
```

1.11.3 코드 해설

1. **4가지 데이터셋**: 선형 분리, 원형, XOR, 반달 - 각각 다른 구조를 가져 서로 다른 알고리즘에 유리
2. **5가지 알고리즘**: 로지스틱 회귀, k-NN, SVM, 결정 트리, 랜덤 포레스트 - 각각 다른 귀납적 편향(inductive

bias)을 가짐
3. **Stratified 5-Fold 교차검증**으로 공정한 성능 비교
4. **핵심 관찰**: 선형 데이터에서는 로지스틱 회귀가, 원형 데이터에서는 SVM이, XOR에서는 결정 트리가, 반달에서는 k-NN이 각각 유리한 경향을 보임

1.12 실습: 교차검증 비교

1.12.1 실습 목표

- Hold-Out, K-Fold, Stratified K-Fold, LOO 네 가지 교차검증 전략을 비교한다
- 각 전략의 성능 추정 분산(안정성)을 실험적으로 측정한다
- K값에 따른 편향-분산 트레이드오프를 확인한다

1.12.2 전체 코드

```
# ============================================================
# 03_cross_validation_demo.py
# 교차 검증 (Cross-Validation) 전략 비교 데모
#
# 전략: Hold-Out, K-Fold, Stratified K-Fold, Leave-One-Out (LOO)
# ============================================================

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

from sklearn.datasets import load_iris, make_classification
from sklearn.model_selection import (
    train_test_split, KFold, StratifiedKFold,
    LeaveOneOut, cross_val_score
)
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

# --- 한글 폰트 설정 ---
plt.rcParams['font.family'] = 'Malgun Gothic'
plt.rcParams['axes.unicode_minus'] = False

# --- 시드 고정 ---
np.random.seed(42)


# ============================================================
# 1. 성능 추정의 분산 비교
# ============================================================
def plot_variance_comparison():
    """
    각 교차 검증 전략을 여러 번 반복하여,
    성능 추정치의 분산(안정성)을 비교한다.
    """
    X, y = make_classification(
        n_samples=150, n_features=10, n_informative=5,
```

```
    n_redundant=2, n_classes=2, class_sep=1.0,
    random_state=42
)

model = make_pipeline(StandardScaler(),
                      LogisticRegression(max_iter=1000))

n_repeats = 50
results = {}

# --- Hold-Out ---
print("  Hold-Out 평가 중...")
holdout_scores = []
for i in range(n_repeats):
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=i
    )
    model.fit(X_train, y_train)
    score = model.score(X_test, y_test)
    holdout_scores.append(score)
results["Hold-Out\n(80:20)"] = holdout_scores

# --- K-Fold (K=5) ---
print("  K-Fold 평가 중...")
kfold_scores = []
for i in range(n_repeats):
    cv = KFold(n_splits=5, shuffle=True, random_state=i)
    scores = cross_val_score(model, X, y, cv=cv)
    kfold_scores.append(scores.mean())
results["K-Fold\n(K=5)"] = kfold_scores

# --- Stratified K-Fold (K=5) ---
print("  Stratified K-Fold 평가 중...")
skfold_scores = []
for i in range(n_repeats):
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=i)
    scores = cross_val_score(model, X, y, cv=cv)
    skfold_scores.append(scores.mean())
results["Stratified\nK-Fold\n(K=5)"] = skfold_scores

# --- K-Fold (K=10) ---
print("  K-Fold (K=10) 평가 중...")
kfold10_scores = []
for i in range(n_repeats):
    cv = KFold(n_splits=10, shuffle=True, random_state=i)
    scores = cross_val_score(model, X, y, cv=cv)
    kfold10_scores.append(scores.mean())
results["K-Fold\n(K=10)"] = kfold10_scores

# --- Leave-One-Out ---
print("  Leave-One-Out 평가 중...")
loo = LeaveOneOut()
loo_scores = cross_val_score(model, X, y, cv=loo)
```

```
    loo_mean = loo_scores.mean()
    results["LOO"] = [loo_mean] * n_repeats

    # --- 시각화 ---
    fig, ax = plt.subplots(figsize=(12, 6))
    labels = list(results.keys())
    data = [results[k] for k in labels]

    bp = ax.boxplot(data, labels=labels, patch_artist=True,
                    widths=0.6, showmeans=True,
                    meanprops=dict(marker='D', markerfacecolor='red',
                                   markersize=6))

    colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#96CEB4', '#FFEAA7']
    for patch, color in zip(bp['boxes'], colors):
        patch.set_facecolor(color)
        patch.set_alpha(0.7)

    ax.set_ylabel('정확도 (Accuracy)', fontsize=12)
    ax.set_title('교차 검증 전략별 성능 추정 분포 (50회 반복)',
                 fontsize=12, fontweight='bold')
    ax.grid(axis='y', alpha=0.3)

    # 통계 출력
    print("\n[성능 추정 통계 요약]")
    for name, scores in results.items():
        clean_name = name.replace('\n', ' ')
        print(f"  {clean_name:<25}: 평균={np.mean(scores):.4f}, "
              f"표준편차={np.std(scores):.4f}")

    plt.tight_layout()
    plt.savefig('03_variance_comparison.png', dpi=150, bbox_inches='tight')
    plt.show()
    print("[저장 완료] 03_variance_comparison.png")
    return results


# ============================================================
# 2. K값에 따른 편향-분산 트레이드오프
# ============================================================
def plot_k_tradeoff():
    """
    K-Fold에서 K값의 변화에 따른 성능 추정의 편향과 분산을 분석한다.
    """
    X, y = make_classification(
        n_samples=100, n_features=10, n_informative=5,
        n_redundant=2, n_classes=2, class_sep=1.0,
        random_state=42
    )

    model = make_pipeline(StandardScaler(),
                          LogisticRegression(max_iter=1000))
```

```
k_values = [2, 3, 5, 7, 10, 15, 20, 50, 100]
means = []
stds = []
n_repeats = 30

for k in k_values:
    if k >= len(X):
        cv = LeaveOneOut()
        scores = cross_val_score(model, X, y, cv=cv)
        means.append(scores.mean())
        stds.append(0.0)
    else:
        repeat_means = []
        for i in range(n_repeats):
            cv = KFold(n_splits=k, shuffle=True, random_state=i)
            scores = cross_val_score(model, X, y, cv=cv)
            repeat_means.append(scores.mean())
        means.append(np.mean(repeat_means))
        stds.append(np.std(repeat_means))

fig, axes = plt.subplots(1, 2, figsize=(15, 6))

ax1 = axes[0]
ax1.errorbar(k_values, means, yerr=stds, fmt='o-',
             color='#4285F4', linewidth=2, markersize=8,
             capsize=5, capthick=2,
             label='평균 정확도 +/- 표준편차')
ax1.set_xlabel('K (폴드 수)', fontsize=12)
ax1.set_ylabel('평균 정확도', fontsize=12)
ax1.set_title('K값에 따른 평균 성능 추정\n(K 증가 -> 편향 감소)',
              fontsize=12, fontweight='bold')
ax1.set_xscale('log')
ax1.set_xticks(k_values)
ax1.set_xticklabels([str(k) for k in k_values])
ax1.grid(True, alpha=0.3)
ax1.legend(fontsize=10)

ax2 = axes[1]
ax2.plot(k_values, stds, 'o-', color='#EA4335', linewidth=2,
         markersize=8, label='성능 추정의 표준편차')
ax2.fill_between(k_values, 0, stds, alpha=0.2, color='#EA4335')
ax2.set_xlabel('K (폴드 수)', fontsize=12)
ax2.set_ylabel('표준편차', fontsize=12)
ax2.set_title('K값에 따른 성능 추정 분산\n(K 증가 -> 분산 증가 경향)',
              fontsize=12, fontweight='bold')
ax2.set_xscale('log')
ax2.set_xticks(k_values)
ax2.set_xticklabels([str(k) for k in k_values])
ax2.grid(True, alpha=0.3)
ax2.legend(fontsize=10)

plt.suptitle('K-Fold의 K값에 따른 편향-분산 트레이드오프',
             fontsize=14, fontweight='bold', y=1.02)
```

```
    plt.tight_layout()
    plt.savefig('03_k_tradeoff.png', dpi=150, bbox_inches='tight')
    plt.show()
    print("[저장 완료] 03_k_tradeoff.png")


# ============================================================
# 3. Iris 데이터셋 실전 예제
# ============================================================
def plot_iris_cv_comparison():
    """Iris 데이터셋에서 다양한 CV 전략의 실제 성능을 비교한다."""
    iris = load_iris()
    X, y = iris.data, iris.target

    model = make_pipeline(StandardScaler(),
                          LogisticRegression(max_iter=1000))

    print(f"\n[Iris 데이터셋 교차 검증 결과]")
    print(f"데이터 크기: {X.shape[0]}개, 특성 수: {X.shape[1]}개")

    all_scores = {}

    # Hold-Out (10회 반복)
    holdout_scores = []
    for i in range(10):
        X_tr, X_te, y_tr, y_te = train_test_split(
            X, y, test_size=0.3, random_state=i, stratify=y
        )
        model.fit(X_tr, y_tr)
        holdout_scores.append(model.score(X_te, y_te))
    all_scores['Hold-Out'] = holdout_scores

    # 5-Fold
    cv5 = KFold(n_splits=5, shuffle=True, random_state=42)
    all_scores['5-Fold'] = cross_val_score(model, X, y, cv=cv5)

    # Stratified 5-Fold
    cv5s = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    all_scores['Stratified\n5-Fold'] = cross_val_score(model, X, y, cv=cv5s)

    # 10-Fold
    cv10 = KFold(n_splits=10, shuffle=True, random_state=42)
    all_scores['10-Fold'] = cross_val_score(model, X, y, cv=cv10)

    # LOO
    loo = LeaveOneOut()
    all_scores['LOO'] = cross_val_score(model, X, y, cv=loo)

    # 시각화
    fig, ax = plt.subplots(figsize=(12, 6))
    labels = list(all_scores.keys())
    data = [all_scores[k] for k in labels]
```

```
    bp = ax.boxplot(data, labels=labels, patch_artist=True,
                    widths=0.5, showmeans=True,
                    meanprops=dict(marker='D', markerfacecolor='red',
                                   markersize=7))

    colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#96CEB4', '#FFEAA7']
    for patch, color in zip(bp['boxes'], colors):
        patch.set_facecolor(color)
        patch.set_alpha(0.7)

    for i, d in enumerate(data):
        mean_val = np.mean(d)
        ax.text(i + 1, mean_val + 0.005, f'{mean_val:.3f}',
                ha='center', fontsize=10, fontweight='bold', color='red')

    ax.set_ylabel('정확도 (Accuracy)', fontsize=12)
    ax.set_title('Iris 데이터셋: 교차 검증 전략별 성능 비교',
                 fontsize=13, fontweight='bold')
    ax.grid(axis='y', alpha=0.3)

    plt.tight_layout()
    plt.savefig('03_iris_cv_comparison.png', dpi=150, bbox_inches='tight')
    plt.show()
    print("[저장 완료] 03_iris_cv_comparison.png")


# ============================================================
# 메인 실행
# ============================================================
if __name__ == "__main__":
    print("=" * 60)
    print("교차 검증 (Cross-Validation) 전략 비교 데모")
    print("=" * 60)

    print("\n[1/3] 성능 추정 분산 비교...")
    results = plot_variance_comparison()

    print("\n[2/3] K값에 따른 편향-분산 트레이드오프...")
    plot_k_tradeoff()

    print("\n[3/3] Iris 데이터셋 실전 비교...")
    plot_iris_cv_comparison()

    print("\n최종 요약: 교차 검증 전략 선택 가이드")
    print("  - 기본 권장: Stratified K-Fold (K=5 또는 K=10)")
    print("  - 대용량: Hold-Out 또는 3-Fold")
    print("  - 소용량: LOO 또는 Repeated K-Fold")
    print("  - 실무적 최적점: K=5~10")
```

1.12.3 코드 해설

1. **분산 비교 실험**: 동일한 데이터에 대해 각 CV 전략을 50회 반복하여, 성능 추정치의 분산(안정성)을 비교
2. **K값 트레이드오프**: K를 2부터 100(=LOO)까지 변화시키며 편향과 분산의 변화를 관찰

3. **Iris 실전 예제**: 실제 데이터셋에서 각 CV 전략의 성능 추정 결과를 박스플롯으로 비교
4. **핵심 관찰**: Hold-Out은 분산이 가장 크고, Stratified K-Fold가 가장 안정적인 추정을 제공함

1.13 응용사례 심화

1.13.1 의료 (Healthcare)

적용 분야: 질병 진단, 의료 영상 분석, 신약 개발, 환자 예후 예측

사례: 유방암 진단 시스템

Wisconsin Breast Cancer Dataset을 활용한 이진 분류 문제이다. 세침 흡인 생검(FNA) 이미지에서 추출한 세포핵의 30개 특성을 기반으로 악성/양성을 분류한다.

항목	내용
데이터	569개 샘플, 30개 특성
클래스	악성(37.3%), 양성(62.7%)
핵심 지표	재현율(Recall) - 암 환자를 놓치지 않는 것이 핵심
성능	로지스틱 회귀만으로도 정확도 97~98%, AUC 0.99 수준

의료 AI의 핵심 고려사항: - FN(위음성) 최소화가 생명과 직결 - 정확도보다 재현율과 F1-Score를 중시 - 모델의 해석 가능성(Explainability)이 중요 - 의사가 이해할 수 있어야 함 - 규제 요건 충족 (FDA 승인 등)

1.13.2 금융 (Finance)

적용 분야: 신용 평가, 사기 탐지, 알고리즘 트레이딩, 리스크 관리

사례: 신용카드 사기 탐지

항목	내용
문제 유형	극심한 클래스 불균형 (사기 < 1%)
핵심 과제	정상 거래를 사기로 잘못 판단(FP) 최소화 + 실제 사기 탐지(Recall)
주요 기법	이상치 탐지(Isolation Forest), 오버샘플링(SMOTE), 앙상블
특이점	실시간 처리 필요, 패턴이 지속적으로 변화

금융 ML의 핵심 고려사항: - 시계열 특성: Time Series Split 사용, 미래 데이터 누출 방지 - 클래스 불균형: SMOTE, 비용 민감 학습(Cost-sensitive Learning) - 해석 가능성: 규제 당국에 모델 설명 필요 (Basel III, GDPR) - 실시간 추론 성능 요구

1.13.3 제조 (Manufacturing)

적용 분야: 품질 검사, 예측 정비(Predictive Maintenance), 공정 최적화

사례: 예측 정비 시스템

항목	내용
문제 유형	장비 고장 시점 예측 (회귀) 또는 고장 여부 예측 (분류)
데이터	IoT 센서 데이터 (진동, 온도, 압력, 소음)
주요 기법	Random Forest, LSTM, Survival Analysis
효과	비계획 정지 시간 30~50% 감소, 유지보수 비용 절감

제조 ML의 핵심 고려사항: - 센서 데이터의 시계열 특성 - 고장 데이터가 매우 희소 (불균형 문제) - Edge Computing으로 공장 내 실시간 추론 - 도메인 전문가(공정 엔지니어)와의 협업이 필수

1.13.4 NLP (Natural Language Processing)

적용 분야: 기계 번역, 감성 분석, 텍스트 요약, 질의응답, 챗봇

사례: 감성 분석 (Sentiment Analysis)

항목	내용
문제 유형	텍스트 분류 (긍정/부정/중립)
데이터	리뷰, 소셜 미디어 게시글, 뉴스 기사
전통적 접근	TF-IDF + SVM/로지스틱 회귀
현대적 접근	BERT, GPT 등 사전학습 언어 모델 Fine-tuning

NLP ML의 핵심 고려사항: - 텍스트 전처리: 토큰화, 불용어 제거, 정규화 - 피처 표현: BoW, TF-IDF, Word2Vec, BERT Embedding - 전이 학습(Transfer Learning): 사전학습 모델을 특정 태스크에 미세 조정 - 다국어 처리: 한국어 형태소 분석의 특수성

1.14 핵심 요약표 + 복습 질문 10개

1.14.1 핵심 요약표

번호	핵심 개념	요약
1	**머신러닝 정의**	데이터로부터 패턴을 자동 학습하여 예측하는 기술. Mitchell(1997)의 T-P-E 프레임워크
2	**지도학습**	입력-정답 쌍으로 학습. 분류(이산)와 회귀(연속)로 구분
3	**비지도학습**	정답 없이 데이터 구조 발견. 군집화, 차원 축소, 이상 탐지
4	**강화학습**	보상 최대화 행동 전략 학습. 에이전트-환경 상호작용
5	**AI ⊃ ML ⊃ DL**	딥러닝은 머신러닝의 부분집합, 머신러닝은 AI의 부분집합
6	**편향-분산 트레이드오프**	$EPE = \text{Bias}^2 + \text{Variance} + \sigma^2$. 모델 복잡도의 최적 균형점 탐색
7	**NFL 정리**	모든 문제에 보편적으로 최상인 알고리즘은 없다 (Wolpert & Macready, 1997)
8	**교차검증**	데이터를 K개 폴드로 나눠 반복 평가. Stratified K-Fold(K=5~10) 권장
9	**ML 파이프라인**	문제정의 → 데이터수집 → 전처리 → EDA → 모델링 → 평가 → 튜닝 → 배포

번호	핵심 개념	요약
10	**데이터 품질**	"Garbage In, Garbage Out". 피처 엔지니어링이 알고리즘 선택보다 중요할 수 있음
11	**과적합**	학습 데이터에 과도하게 맞춤. 높은 학습 성능, 낮은 테스트 성능
12	**과소적합**	데이터 패턴을 포착 못함. 학습/테스트 모두 낮은 성능
13	**귀납적 편향**	학습 알고리즘이 새 데이터에 일반화하기 위한 사전 가정 (Mitchell, 1997)
14	**VC 차원**	모델 복잡도의 측정 지표. 가설 공간이 shatter할 수 있는 최대 점 수 (Vapnik, 1995)
15	**12가지 교훈**	Domingos(2012)의 실무 가이드: 일반화, 피처, 데이터, 앙상블, 상관 vs 인과

1.14.2 핵심 수식 정리

수식	이름	의미
$EPE = \text{Bias}^2 + \text{Var} + \sigma^2$	편향-분산 분해	총 오차의 세 가지 원천
$\text{Bias}(x) = f(x) - E[\hat{f}(x)]$	편향	모델 가정에 의한 체계적 오차
$\text{Var}(x) = E[(\hat{f}(x) - E[\hat{f}(x)])^2]$	분산	데이터 변동에 의한 예측 불안정
$\sum_f P(\mathbf{d}_m^y \mid f, m, a_1) = \sum_f P(\mathbf{d}_m^y \mid f, m, a_2)$	NFL 정리	모든 알고리즘의 평균 성능 동일
$R(f) \leq R_{\text{emp}}(f) + \Phi(\text{VCdim}, n)$	구조적 위험 최소화	일반화 오류 상한
$f^*(x) = E[Y \mid X = x]$	최적 예측기 (회귀)	MSE를 최소화하는 예측

1.14.3 복습 질문 10개

질문 1: Tom Mitchell(1997)의 머신러닝 정의에서 T, P, E는 각각 무엇을 의미하는가? 스팸 필터 문제에 적용하여 구체적으로 설명하시오.

질문 2: 지도학습, 비지도학습, 강화학습의 차이를 데이터 형태, 학습 목표, 피드백 방식의 세 가지 관점에서 비교하시오.

질문 3: 편향-분산 분해 공식 $EPE = \text{Bias}^2 + \text{Variance} + \sigma^2$에서 각 항의 의미를 설명하고, 모델 복잡도가 증가할 때 각 항이 어떻게 변하는지 서술하시오.

질문 4: No Free Lunch 정리의 핵심 내용을 설명하고, 이 정리가 머신러닝 실무에 주는 시사점을 3가지 이상 서술하시오.

질문 5: Hold-Out, K-Fold, Stratified K-Fold, LOO 네 가지 교차검증 전략의 장단점을 비교하고, 데이터 크기에 따라 어떤 전략을 선택해야 하는지 설명하시오.

질문 6: 과적합(Overfitting)과 과소적합(Underfitting)의 원인과 해결 방법을 각각 3가지 이상 서술하시오. Domingos(2012)의 "Overfitting Has Many Faces"와 연결하여 설명하시오.

질문 7: Vapnik(1995)의 VC 차원(VC Dimension)이란 무엇인지 정의하고, 2차원 선형 분류기의 VC 차원이 3인 이유를 직관적으로 설명하시오.

질문 8: 머신러닝 파이프라인의 8단계를 순서대로 나열하고, 각 단계에서 수행하는 핵심 활동을 간략히 설명하시오. 특히 데이터 전처리 단계에서 주의해야 할 데이터 누출(Data Leakage) 문제를 설명하시오.

질문 9: ESL(Hastie et al., 2009)에서 제시한 최소제곱법과 k-NN의 비교를 통해, 편향-분산 트레이드오프를 직관적으로 설명하시오. 각 방법의 가정, 편향, 분산 수준을 비교하시오.

질문 10: Domingos(2012)의 12가지 교훈 중 "More Data Beats a Cleverer Algorithm"과 "Feature Engineering Is the Key"를 설명하고, 이 두 교훈이 실무에서 어떻게 적용되는지 구체적 예시와 함께 서술하시오.

참고문헌

1. Domingos, P. (2012). A Few Useful Things to Know About Machine Learning. *Communications of the ACM*, 55(10), 78-87.
2. Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.
3. Wolpert, D. H., & Macready, W. G. (1997). No Free Lunch Theorems for Optimization. *IEEE Transactions on Evolutionary Computation*, 1(1), 67-82.
4. Vapnik, V. N. (1995). *The Nature of Statistical Learning Theory*. Springer-Verlag.
5. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.

다음 장 예고: 2장에서는 지도학습의 첫 번째 알고리즘인 **선형 회귀(Linear Regression)**를 다룬다. 최소제곱법의 수학적 유도, 경사하강법, 정규화(Ridge/Lasso) 등을 학습한다.